FORWARD DEPLOYED / BACKEND SYSTEMS

INVARIANT OPS

05

PROPOSED RESEARCH / NOT YET VERIFIED

THE ENGINEERING QUESTION

Which transaction and delivery patterns preserve business invariants when external integrations are unreliable?

PROBLEM DEFINITION AND RESEARCH CONTRIBUTION

A failure prone procurement workflow built to demonstrate business correctness through retries, crashes, stale schemas, duplicate messages and partial transactions.

- The key is preserving business truth, not showcasing infrastructure complexity for its own sake.
- This is highly relevant for forward deployed and platform engineering if it is tied to explicit invariants.
- It should sit beside anonymised real delivery case studies rather than replacing them.

ARCHITECTURE AND IMPLEMENTATION SPECIFICATION

1 STAGE **External systems** → 2 STAGE **Durable events** → 3 STAGE **Domain workflow** → 4 STAGE **Reconciliation**

- Durable event flow using PostgreSQL transaction boundaries, outbox/inbox handling and idempotent consumers.
- Explicit business invariants, manual reconciliation paths, role boundaries and audit visibility.
- Local deployment scripts and deterministic failure injection so reviewers can replay the bad cases.

ENGINEERING DECISIONS TO MAKE EXPLICIT

The design review must record data contracts, state transitions, failure handling and why the selected baseline is defensible. Every subsystem should have an interface specification and a reproducible test route.

DESIGN BOUNDARIES AND ASSUMPTIONS

⚠ Exactly once business outcomes depend on application invariants, not simply middleware settings.

⚠ Synthetic workflows cannot substitute for evidence from real customer delivery.

ⓘ Scope is proposed unless explicitly identified as documented existing work. All measured claims require reproducible evidence.

INVARIANT OPS / PROTOCOL / ACCEPTANCE / REVIEW

EXPERIMENTS, VERIFICATION & PUBLICATION

05

PROPOSED RESEARCH / NOT YET VERIFIED

EXPERIMENTAL METHODOLOGY

01 Model purchase orders, goods receipts, invoice approval and synthetic legacy APIs with explicit correctness rules.

02 Inject crashes at commit, ack, retry, schema version and restart boundaries.

03 Measure reconciliation, recovery latency and duplicate suppression under repeated failure campaigns.

MEASUREMENT AND ACCEPTANCE EVIDENCE

- No duplicate payment/posting under retry and replay
- State reconciliation after crash and restart
- Recovery latency and backlog under injected failure

REQUIRED REPOSITORY ARTEFACTS

- Executable invariant test suite
- Failure trace library and local deployment guide
- Architecture record plus anonymised case study linkage

TECHNICAL REVIEW QUESTIONS

Which assumptions could invalidate the result? How do the baselines behave under the same conditions? What is the strongest negative case? Which variables, seeds, workloads, software versions and machine properties must be retained for independent reproduction?

VERIFICATION MAPPING

For each acceptance claim, record the test identifier, input configuration, observed result, pass/fail definition and an independent rerun procedure. Failed or inconclusive cases remain in the report rather than being silently excluded.

PUBLICATION DISCIPLINE

Release original source, configuration files, test harnesses, raw measurements and scripts where rights permit. Distinguish simulation outcomes from physical validation and completed achievements from planned evaluation. Results must not be inferred from targets.